that can extract relevant features from images and subsequently compare them using their corresponding features.

In the previous chapters, we learned that we can use convolutional neural networks (CNNs) to extract meaningful features for an image. This chapter will use our understanding of CNNs to train (jointly) a *visual embedding layer*. In this chapter's context, *visual embedding* refers to the last fully connected layer (prior to a loss layer) appended to a CNN. *Joint training* refers to training both the embedding layer and the CNN parameters jointly.

This chapter explores the nuts and bolts of training and using visual embeddings for large-scale, image-based query-retrieval systems such as applications of visual embeddings (see figure 10.1). To perform this task, we first need to project (*embed*) our database of images onto a *vector space* (*embedding*). This way, comparisons between images can be performed by measuring their pairwise distances in this embedding space. This is the high-level idea of visual embedding systems.

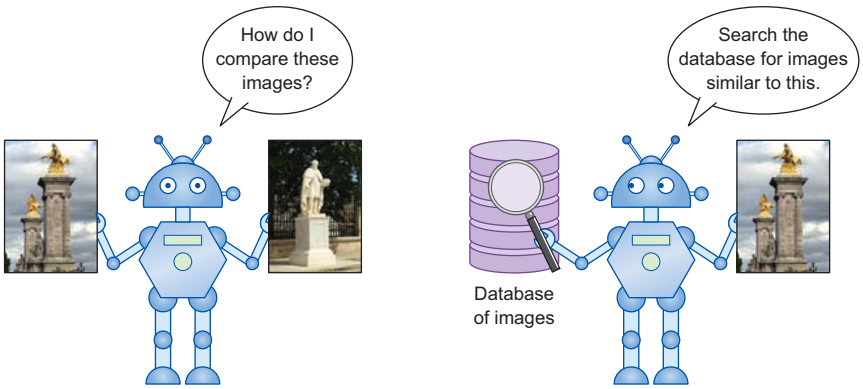

Figure 10.1 Example applications we encounter in everyday life when working with images: a machine comparing two images (left); querying the database to find images similar to the input image (right). Comparing two images is a non-trivial task and is key to many applications relating to meaningful image search.

DEFINITION An *embedding* is a vector space, typically of lower dimension than the input space, which preserves relative dissimilarity (in the input space). We use the terms *vector space* and *embedding space* interchangeably. In the context of this chapter, the last fully connected layer of a trained CNN is this vector (embedding) space. As an example, a fully connected layer of 128 neurons corresponds to a vector space of 128 dimensions.

For a reliable comparison among images, the embedding function needs to capture a desired input similarity measure. This embedding function can be learned using various approaches; one of the popular ways is to use a deep CNN. Figure 10.2 illustrates a high-level process of using CNNs to create an embedding.